UNIVERSIDAD ANTONIO NARIÑO

FACULTAD DE INGENIERÍA · PROGRAMA DE INGENIERÍA ELECTRÓNICA

SEDE IBAGUÉ — CAPITAL MUSICAL DE COLOMBIA

ANEXO DE CÓDIGO FUENTE

ARQUITECTURA DE SOFTWARE DSP, ALGORITMOS DE SÍNTESIS Y MOTOR DE AUDIO EN TIEMPO REAL

Aurora By Juan Esteban Vera — Yamaha MONTAGE DSP Edition

Autor y Desarrollador:

Juan Esteban Vera Rodríguez
Investigador Principal · Estudiante de Ingeniería Electrónica
Universidad Antonio Nariño · Sede Ibagué

Proyecto Institucional:
Repositorio de Código Fuente y Registro de Propiedad Intelectual
Dirección Nacional de Derecho de Autor (DNDA)

Identificador Documental:
ANX-SRC-002 · Especificación de Software y Códigos de Ingeniería

Ibagué, Tolima, Colombia · 2026

Contents

1 Arquitectura Global del Software **2**
1.1 Diagrama de Componentes y Flujo de Señales 2

2 Módulo 1: Motor DSP en Tiempo Real (`synth/engine.py`) **2**
2.1 Estructura Inmutable de Parámetros 2
2.2 Ecuación de Afinación y Supresión de Aliasing PolyBLEP 3
2.3 Generador de Formas de Onda y Modulación FM-X 3
2.4 Filtro Analógico Virtual TPT SVF de 2 Polos 3
2.5 Ciclo de Vida de las Voces y Envolvente ADSR Vectorizada 4
2.6 El Motion Control Super Knob y Efectos Master 5

3 Módulo 2: Procesador Vocal & Autotune (`synth/vocal.py`) **5**
3.1 Detección de Tono Fundamental (Pitch Detector) 5
3.2 Cuantización de Escala y Corrector Granular 6

4 Módulo 3: Banco de Presets Yamaha (`synth/presets.py`) **6**

5 Módulo 4: Consola de Trabajo GUI (`synth/app.py`) **7**
5.1 El Componente Gráfico Super Knob 7
5.2 Teclado Extendido de 25 Teclas Interactivas 8

6 Módulo 5: Suite de Pruebas Unitarias (`tests/test_engine.py`) **8**

7 Punto de Entrada Principal (`main.py`) **9**

1 Arquitectura Global del Software

El sistema **Aurora By Juan Esteban Vera** fue diseñado bajo un patrón arquitectónico desacoplado y orientado a objetos, garantizando que el cómputo matemático de audio en tiempo real opere de forma estrictamente asíncrona e independiente del hilo principal de renderizado gráfico (GUI).

1.1 Diagrama de Componentes y Flujo de Señales

- **Capa de Síntesis y DSP (`synth/engine.py`):** Aloja las estructuras inmutables de parámetros (`SynthParameters`), el generador de formas de onda con PolyBLEP, la modulación FM-X, el filtro TPT SVF, la envolvente ADSR vectorizada, y la cadena de efectos (Drive, Stereo Delay, Reverb Schroeder).
- **Capa de Procesamiento Vocal (`synth/vocal.py`):** Contiene el detector de tono fundamental (FFT / autocorrelación normalizada) y el corrector granular de tono afinado en tiempo real según escalas modales.
- **Capa de Presets de Fábrica (`synth/presets.py`):** Define los bancos de memoria tímbrica calibrados con base en los instrumentos clásicos de Yamaha (DX7, CS-80 y MONTAGE).
- **Capa de Interfaz de Usuario y Secuenciador (`synth/app.py`):** Implementa la consola de control con el teclado extendido de 25 teclas, el dial macro Super Knob, el secuenciador de 16 pasos, el osciloscopio en vivo y los medidores balísticos.
- **Capa de Validación Automatizada (`tests/test_engine.py`):** Suite de 13 pruebas unitarias para verificación de estabilidad numérica y latencia.

2 Módulo 1: Motor DSP en Tiempo Real (`synth/engine.py`)

2.1 Estructura Inmutable de Parámetros

La clase `SynthParameters` encapsula los 17 parámetros de control tímbrico y espacial mediante el decorador `@dataclass(frozen=True)`, asegurando concurrencia *thread-safe* sin bloqueos (*lock-free*):

```
@dataclass(frozen=True)
class SynthParameters:
    # Osciladores principales
    waveform_1: str = "sine"
    waveform_2: str = "triangle"
    osc2_mix: float = 0.18
    detune_cents: float = 0.0
    osc2_semitones: float = 0.0

    # Sintesis FM (Yamaha DX / FM-X style)
    fm_depth: float = 0.0

    # Envolvente ADSR
    attack: float = 0.012
    decay: float = 0.18
    sustain: float = 0.72
    release: float = 0.32

    # Filtro & Resonancia (Yamaha CS-80 style)
    cutoff: float = 6500.0
    resonance: float = 0.0

    # Procesador de Efectos (Yamaha MONTAGE Insertion FX)
```

```
    drive: float = 0.0
    delay_time: float = 0.25
    delay_feedback: float = 0.35
    delay_mix: float = 0.0

    # Modulacion, espacio y volumen
    vibrato_rate: float = 5.0
    vibrato_depth: float = 0.0
    master_gain: float = 0.35
    reverb_mix: float = 0.18
    reverb_room: float = 0.55

    # Mando maestro Motion Control (Yamaha MONTAGE Super Knob)
    super_knob: float = 0.0
```

2.2 Ecuación de Afinación y Supresión de Aliasing PolyBLEP

La afinación temperada igual sigue la norma ISO 16, mientras que el algoritmo PolyBLEP inserta correcciones polinómicas para suavizar las discontinuidades en los bordes de fase:

```
def midi_to_hz(note: int | float) -> float:
    return 440.0 * 2.0 ** ((note - 69) / 12.0)

def _poly_blep(phase: np.ndarray, increment: float) -> np.ndarray:
    result = np.zeros_like(phase)
    first = phase < increment
    t = phase[first] / increment
    result[first] = 2.0 * t - t * t - 1.0
    last = phase > 1.0 - increment
    t = (phase[last] - 1.0) / increment
    result[last] = t * t + 2.0 * t + 1.0
    return result
```

2.3 Generador de Formas de Onda y Modulación FM-X

El renderizador vectorial genera formas de onda sinusoidales, triangulares, cuadradas y diente de sierra, incorporando modulación de fase para la síntesis FM:

```
def _render_waveform(waveform: str, phase: np.ndarray, increment: float, phase_mod: np.
    ndarray | None = None) -> np.ndarray:
    total_phase = phase if phase_mod is None else (phase + phase_mod)
    total_phase = np.mod(total_phase, 1.0)

    if waveform == "sine":
        return np.sin(2.0 * np.pi * total_phase)
    if waveform == "triangle":
        saw = 2.0 * total_phase - 1.0
        return 2.0 * np.abs(saw) - 1.0
    if waveform == "saw":
        naive = 2.0 * total_phase - 1.0
        return naive - _poly_blep(total_phase, increment)
    if waveform == "square":
        naive = np.where(total_phase < 0.5, 1.0, -1.0)
        corr_0 = _poly_blep(total_phase, increment)
        corr_half = _poly_blep(np.mod(total_phase + 0.5, 1.0), increment)
        return naive + corr_0 - corr_half
    if waveform == "noise":
        return np.random.uniform(-1.0, 1.0, size=phase.shape)
    return np.sin(2.0 * np.pi * total_phase)
```

2.4 Filtro Analógico Virtual TPT SVF de 2 Polos

Filtro de variable de estado discretizado mediante Transformada Bilineal con prewarping de frecuencia:

```
def _process_svf(signal: np.ndarray, cutoff_hz: float, resonance: float, sample_rate: int
    , s1: float, s2: float):
    cutoff = np.clip(cutoff_hz, 40.0, sample_rate * 0.45)
    res = np.clip(resonance, 0.0, 0.95)
    g = math.tan(math.pi * cutoff / sample_rate)
    k = 2.0 * (1.0 - res)
    a1 = 1.0 / (1.0 + g * (g + k))
    a2 = g * a1
    a3 = g * a2

    out = np.empty_like(signal)
    for n in range(signal.shape[0]):
        v0 = signal[n]
        v1 = a1 * s1 + a2 * (v0 - s2)
        v2 = s2 + a3 * (v0 - s2) + a2 * s1
        s1 = 2.0 * v1 - s1
        s2 = 2.0 * v2 - s2
        out[n] = math.tanh(v2)
    return out, s1, s2
```

2.5 Ciclo de Vida de las Voces y Envolvente ADSR Vectorizada

Cada voz activa calcula su ganancia temporal en función de las 4 etapas ADSR:

```
class ActiveVoice:
    def __init__(self, note: int, sample_rate: int, params: SynthParameters):
        self.note = note
        self.sample_rate = sample_rate
        self.params = params
        self.base_frequency = midi_to_hz(note)
        self.phase_1 = 0.0
        self.phase_2 = 0.0
        self.lfo_phase = 0.0
        self.filter_s1_left = 0.0
        self.filter_s2_left = 0.0
        self.filter_s1_right = 0.0
        self.filter_s2_right = 0.0
        self.age_samples = 0
        self.released = False
        self.release_start_sample = 0
        self.release_start_level = 1.0
        self.is_finished = False

    def render(self, num_samples: int, params: SynthParameters) -> np.ndarray:
        # Generacion de incrementos angulares
        dt1 = self.base_frequency / self.sample_rate
        osc2_ratio = 2.0 ** (params.osc2_semitones / 12.0 + params.detune_cents / 1200.0)
        dt2 = (self.base_frequency * osc2_ratio) / self.sample_rate

        phase_1_arr = np.mod(self.phase_1 + np.arange(num_samples) * dt1, 1.0)
        phase_2_arr = np.mod(self.phase_2 + np.arange(num_samples) * dt2, 1.0)

        # Modulador FM
        modulator = _render_waveform(params.waveform_2, phase_2_arr, dt2)
        fm_term = modulator * (params.fm_depth * 1.8) if params.fm_depth > 0.001 else
            None
        carrier = _render_waveform(params.waveform_1, phase_1_arr, dt1, fm_term)

        # Mezcla balanceada
        raw_voice = carrier * (1.0 - params.osc2_mix * 0.5) + modulator * (params.
            osc2_mix * 0.7)

        # Calculo de Envolvente ADSR
        env = self._calculate_adsr(num_samples, params)
        voice_out = raw_voice * env

        # Filtrado SVF
        filtered, self.filter_s1_left, self.filter_s2_left = _process_svf(
            voice_out, params.cutoff, params.resonance, self.sample_rate,
```

```
            self.filter_s1_left, self.filter_s2_left
        )

        # Avance de acumuladores
        self.phase_1 = (self.phase_1 + num_samples * dt1) % 1.0
        self.phase_2 = (self.phase_2 + num_samples * dt2) % 1.0
        self.age_samples += num_samples

        stereo = np.column_stack((filtered, filtered))
        return stereo
```

2.6 El Motion Control Super Knob y Efectos Master

La función `_effective_params` calcula la interpolación macro entre los parámetros base y el dial Super Knob:

```
def _effective_params(params: SynthParameters) -> SynthParameters:
    knob = float(np.clip(params.super_knob, 0.0, 1.0))
    if knob <= 0.0001:
        return params

    return replace(
        params,
        cutoff=float(np.clip(params.cutoff * (1.0 + 2.0 * knob), 60.0, 15000.0)),
        resonance=float(np.clip(params.resonance + 0.35 * knob, 0.0, 0.94)),
        fm_depth=float(np.clip(params.fm_depth + 0.55 * knob, 0.0, 1.0)),
        drive=float(np.clip(params.drive + 0.45 * knob, 0.0, 1.0)),
        delay_mix=float(np.clip(params.delay_mix + 0.30 * knob, 0.0, 0.85)),
        reverb_mix=float(np.clip(params.reverb_mix + 0.35 * knob, 0.0, 0.85)),
    )
```

3 Módulo 2: Procesador Vocal & Autotune (`synth/vocal.py`)

3.1 Detección de Tono Fundamental (Pitch Detector)

El algoritmo combina filtrado paso-bajo y autocorrelación normalizada mediante FFT (Teorema de Wiener-Khinchin):

```
class PitchDetector:
    def __init__(self, sample_rate: int = 44100, buffer_size: int = 4096):
        self.sample_rate = sample_rate
        self.buffer_size = buffer_size

    def detect_pitch(self, audio_buffer: np.ndarray) -> tuple[float, float]:
        if len(audio_buffer) < self.buffer_size:
            return 0.0, 0.0

        # Normalizacion y ventana Hann
        signal = audio_buffer[-self.buffer_size:]
        signal = signal - np.mean(signal)
        rms = np.sqrt(np.mean(signal**2))
        if rms < 0.008:
            return 0.0, 0.0

        windowed = signal * np.hanning(len(signal))
        # Autocorrelacion via FFT
        fft_data = np.fft.rfft(windowed, n=self.buffer_size * 2)
        corr = np.fft.irfft(fft_data * np.conj(fft_data))[:self.buffer_size]
        corr = corr / (corr[0] + 1e-12)

        # Rango de busqueda: 70 Hz a 1000 Hz
        min_lag = int(self.sample_rate / 1000.0)
        max_lag = int(self.sample_rate / 70.0)

        peak_lag = min_lag + np.argmax(corr[min_lag:max_lag])
        confidence = float(corr[peak_lag])
```

```
        if confidence < 0.35:
            return 0.0, confidence

        detected_hz = float(self.sample_rate / peak_lag)
        return detected_hz, confidence
```

3.2 Cuantización de Escala y Corrector Granular

```
SCALE_PRESETS = {
    "C Mayor": [0, 2, 4, 5, 7, 9, 11],
    "C Menor": [0, 2, 3, 5, 7, 8, 10],
    "A Menor": [0, 2, 3, 5, 7, 8, 10],
    "C Cromatica": list(range(12)),
}

def snap_to_scale(midi_pitch: float, scale_intervals: list[int], root: int = 0) -> float:
    semitone = round(midi_pitch)
    note_class = (semitone - root) % 12
    closest = min(scale_intervals, key=lambda p: min(abs(note_class - p), 12 - abs(
        note_class - p)))
    octave = (semitone - root) // 12
    return float(root + octave * 12 + closest)
```

4 Módulo 3: Banco de Presets Yamaha (`synth/presets.py`)

```
YAMAHA_PRESETS: dict[str, SynthParameters] = {
    "Yamaha DX7 E-Piano": SynthParameters(
        waveform_1="sine", waveform_2="sine", osc2_mix=0.45,
        osc2_semitones=12.0, detune_cents=2.0, fm_depth=0.42,
        attack=0.005, decay=0.65, sustain=0.18, release=0.45,
        cutoff=8200.0, resonance=0.05, vibrato_rate=5.5,
        vibrato_depth=0.04, drive=0.08, delay_mix=0.22,
        reverb_mix=0.32, master_gain=0.38
    ),
    "CS-80 Poly Brass": SynthParameters(
        waveform_1="saw", waveform_2="saw", osc2_mix=0.55,
        osc2_semitones=0.0, detune_cents=12.0, fm_depth=0.0,
        attack=0.085, decay=0.45, sustain=0.75, release=0.55,
        cutoff=3400.0, resonance=0.42, vibrato_rate=4.8,
        vibrato_depth=0.14, drive=0.12, reverb_mix=0.40,
        master_gain=0.34
    ),
    "Montage Super Lead": SynthParameters(
        waveform_1="saw", waveform_2="square", osc2_mix=0.60,
        osc2_semitones=7.0, detune_cents=14.0, fm_depth=0.35,
        attack=0.008, decay=0.25, sustain=0.85, release=0.35,
        cutoff=7800.0, resonance=0.38, drive=0.35,
        delay_mix=0.35, reverb_mix=0.28, super_knob=0.35,
        master_gain=0.32
    ),
    "Ambient Space Pad": SynthParameters(
        waveform_1="triangle", waveform_2="saw", osc2_mix=0.40,
        osc2_semitones=12.0, detune_cents=8.0, fm_depth=0.12,
        attack=0.45, decay=0.85, sustain=0.80, release=1.20,
        cutoff=2800.0, resonance=0.25, delay_mix=0.45,
        reverb_mix=0.65, reverb_room=0.85, master_gain=0.30
    ),
    "Analog Bass 80s": SynthParameters(
        waveform_1="saw", waveform_2="square", osc2_mix=0.50,
        osc2_semitones=-12.0, detune_cents=5.0, fm_depth=0.05,
        attack=0.006, decay=0.22, sustain=0.40, release=0.18,
        cutoff=2200.0, resonance=0.52, drive=0.28,
        delay_mix=0.0, reverb_mix=0.10, master_gain=0.42
    ),
    "Square Bell FM": SynthParameters(
```

```
        waveform_1="sine", waveform_2="square", osc2_mix=0.40,
        osc2_semitones=19.0, detune_cents=0.0, fm_depth=0.68,
        attack=0.002, decay=0.85, sustain=0.05, release=0.60,
        cutoff=9000.0, resonance=0.10, delay_mix=0.30,
        reverb_mix=0.45, master_gain=0.32
    ),
    "Vocal AutoTune FX": SynthParameters(
        waveform_1="saw", waveform_2="sine", osc2_mix=0.30,
        osc2_semitones=0.0, detune_cents=4.0, fm_depth=0.10,
        attack=0.010, decay=0.30, sustain=0.70, release=0.25,
        cutoff=6000.0, resonance=0.20, drive=0.15,
        delay_mix=0.25, reverb_mix=0.30, master_gain=0.35
    )
}
```

5 Módulo 4: Consola de Trabajo GUI (synth/app.py)

5.1 El Componente Gráfico Super Knob

Widget personalizado renderizado mediante primitivas vectoriales de Tkinter Canvas con soporte para arrastre y rueda del mouse:

```
class SuperKnob(tk.Canvas):
    def __init__(self, master, size=150, command=None, **kwargs):
        super().__init__(master, width=size, height=size, bg="#0d141e",
            highlightthickness=0, **kwargs)
        self.size = size
        self.command = command
        self.value = 0.0
        self._dragging = False
        self._last_y = 0

        self.bind("<Button-1>", self._on_press)
        self.bind("<B1-Motion>", self._on_drag)
        self.bind("<ButtonRelease-1>", self._on_release)
        self.bind("<MouseWheel>", self._on_wheel)
        self._draw()

    def set_value(self, val: float, notify: bool = True):
        self.value = float(np.clip(val, 0.0, 1.0))
        self._draw()
        if notify and self.command:
            self.command(self.value)

    def _draw(self):
        self.delete("all")
        cx = cy = self.size / 2.0
        r_outer = self.size * 0.42
        r_inner = self.size * 0.32

        # Anillo de fondo y arco de progreso activo
        self.create_oval(cx - r_outer, cy - r_outer, cx + r_outer, cy + r_outer, outline=
            "#1a2738", width=7)
        start_deg = -225
        extent_deg = -270 * self.value
        self.create_arc(cx - r_outer, cy - r_outer, cx + r_outer, cy + r_outer,
                        start=start_deg, extent=extent_deg, outline="#00e5ff", width=7,
                            style="arc")

        # Dial central y puntero indicador
        self.create_oval(cx - r_inner, cy - r_inner, cx + r_inner, cy + r_inner, fill="
            #131e2d", outline="#253549", width=2)
        angle_rad = math.radians(225 - 270 * self.value)
        px = cx + (r_inner * 0.75) * math.cos(angle_rad)
        py = cy - (r_inner * 0.75) * math.sin(angle_rad)
        self.create_line(cx, cy, px, py, fill="#00e5ff", width=3, capstyle=tk.ROUND)

        # Etiqueta de porcentaje
```

```
        self.create_text(cx, cy, text=f"{int(self.value * 100)}%", fill="#ffffff", font=(
            "Segoe UI", 12, "bold"))
```

5.2 Teclado Extendido de 25 Teclas Interactivas

Implementación de doble octava continua con retroalimentación visual al tacto:

```
KEY_SPECS_25 = [
    # Octava 1 (C3 - B3)
    {"midi_offset": 0, "name": "C", "is_black": False, "key": "Z"},
    {"midi_offset": 1, "name": "C#", "is_black": True,  "key": "S"},
    {"midi_offset": 2, "name": "D", "is_black": False, "key": "X"},
    {"midi_offset": 3, "name": "D#", "is_black": True,  "key": "D"},
    {"midi_offset": 4, "name": "E", "is_black": False, "key": "C"},
    {"midi_offset": 5, "name": "F", "is_black": False, "key": "V"},
    {"midi_offset": 6, "name": "F#", "is_black": True,  "key": "G"},
    {"midi_offset": 7, "name": "G", "is_black": False, "key": "B"},
    {"midi_offset": 8, "name": "G#", "is_black": True,  "key": "H"},
    {"midi_offset": 9, "name": "A", "is_black": False, "key": "N"},
    {"midi_offset": 10, "name": "A#", "is_black": True, "key": "J"},
    {"midi_offset": 11, "name": "B", "is_black": False, "key": "M"},
    # Octava 2 (C4 - C5)
    {"midi_offset": 12, "name": "C", "is_black": False, "key": "Q"},
    {"midi_offset": 13, "name": "C#", "is_black": True, "key": "2"},
    {"midi_offset": 14, "name": "D", "is_black": False, "key": "W"},
    {"midi_offset": 15, "name": "D#", "is_black": True, "key": "3"},
    {"midi_offset": 16, "name": "E", "is_black": False, "key": "E"},
    {"midi_offset": 17, "name": "F", "is_black": False, "key": "R"},
    {"midi_offset": 18, "name": "F#", "is_black": True, "key": "5"},
    {"midi_offset": 19, "name": "G", "is_black": False, "key": "T"},
    {"midi_offset": 20, "name": "G#", "is_black": True, "key": "6"},
    {"midi_offset": 21, "name": "A", "is_black": False, "key": "Y"},
    {"midi_offset": 22, "name": "A#", "is_black": True, "key": "7"},
    {"midi_offset": 23, "name": "B", "is_black": False, "key": "U"},
    {"midi_offset": 24, "name": "C", "is_black": False, "key": "I"},
]
```

6 Módulo 5: Suite de Pruebas Unitarias (`tests/test_engine.py`)

```
class TestAuroraEngine(unittest.TestCase):
    def setUp(self):
        self.engine = AudioEngine(sample_rate=44100, buffer_size=256)

    def tearDown(self):
        self.engine.stop()

    def test_a4_frequency(self):
        self.assertAlmostEqual(midi_to_hz(69), 440.0, places=4)

    def test_note_generates_stereo_audio(self):
        self.engine.note_on(60)
        chunk = self.engine.process_block(256)
        self.assertEqual(chunk.shape, (256, 2))
        self.assertTrue(np.max(np.abs(chunk)) > 0.0)

    def test_release_finishes_voice(self):
        self.engine.note_on(60)
        self.engine.note_off(60)
        # Avanzar el tiempo suficiente para completar el release
        for _ in range(30):
            self.engine.process_block(256)
        self.assertEqual(len(self.engine.voices), 0)

    def test_pitch_detector_reads_a4(self):
        detector = PitchDetector(sample_rate=44100, buffer_size=4096)
        t = np.arange(4096) / 44100.0
```

```
        sine_440 = 0.8 * np.sin(2.0 * np.pi * 440.0 * t)
        freq, conf = detector.detect_pitch(sine_440)
        self.assertAlmostEqual(freq, 440.0, delta=3.0)
        self.assertGreater(conf, 0.25)

    def test_scale_snaps_to_allowed_note(self):
        c_major = SCALE_PRESETS["C Mayor"]
        snapped = snap_to_scale(61.0, c_major) # C# -> C
        self.assertEqual(snapped, 60.0)

    def test_resonant_filter_stability(self):
        noise = np.random.uniform(-1.0, 1.0, 512)
        out, s1, s2 = _process_svf(noise, cutoff_hz=1000.0, resonance=0.92,
                                   sample_rate=44100, s1=0.0, s2=0.0)
        self.assertFalse(np.any(np.isnan(out)))
        self.assertFalse(np.any(np.isinf(out)))
```

7 Punto de Entrada Principal (`main.py`)

```
from synth.app import main

if __name__ == "__main__":
    main()
```